

# How the app turns satellite noise into a pace you can trust

A plain-language tour of the app's construction and the arithmetic behind every number on screen — written so the "why" behind a jumpy split or a slow-to-appear speed is never a mystery.

**Platform** Android & iOS, one codebase    **Phase** 1 of 3 — live tracking & GPX logging

**Reading time** ~10 min

## 01 Executive summary

Simple Activity Tracker is a **Flutter** app — one Dart codebase that compiles to a native Android and a native iOS app, rather than two separate projects kept in sync by hand. It reads your phone's GPS roughly once a second, turns that stream of coordinates into speed, distance, pace and kilometre splits, and writes the raw route to a GPX file you could open in any mapping tool.

The distinctive engineering problem here is not the arithmetic — distance and speed are two lines of trigonometry — it's that **consumer GPS occasionally lies**. A phone can report a location half a kilometre from where you actually are, for one sample, before correcting itself. Unfiltered, that single bad sample would spike your speed to over 1,000 km/h and permanently inflate your total distance. Most of what follows is the app deciding, sample by sample, whether to believe what the GPS just told it.

## 02 Technology & construction

Four building blocks, each doing one job. None of them talk to the phone's GPS hardware directly except one — which is deliberate.

### FRAMEWORK

#### Flutter / Dart

Google's UI toolkit. One source tree compiles to native ARM binaries for both Android and iOS — no shared web view, no separate Swift/Kotlin apps to maintain in parallel.

### GPS ACCESS

#### geolocator

The one package allowed to talk to the OS location APIs directly. Tuned per-platform: `bestForNavigation` accuracy, background updates enabled on iOS via an `Info.plist` entry. On Android it also asks for the phone's location hardware directly rather than the network-assisted provider Android normally prefers — that provider can stall waiting on Wi-Fi even when GPS itself is on and working, which otherwise left a run stuck on "Acquiring GPS..." with Wi-Fi turned off.

### STATE MANAGEMENT

#### Riverpod

A one-way data pipeline: the GPS stream feeds a controller, the controller emits an immutable state object, and every widget on screen just redraws from that. No component reaches backward to mutate another's data.

### TRACK STORAGE

#### GPX 1.1 files

The open standard used by Garmin, Strava, and most mapping software. Every run is written to disk as a standard `.gpx` track — portable, and readable outside this app.

## The data path, GPS fix to screen

### HARDWARE

Phone GPS radio  
  
Combines satellite, Wi-Fi and cell-

### CORE/LOCATION

GeolocatorLocationService  
  
Wraps the OS location

### DOMAIN/TRACKING

MetricsEngine  
  
Pure calculation, no UI

### FEATURES/LIVE\_RUN

LiveRunController  
  
Riverpod controller. Also drives

### UI

Metric tiles  
  
Re-render automatically whenever

tower signals into one position estimate, roughly once per second.

→

API. Nothing outside this file ever sees a raw platform type — the rest of the app only knows a simple lat/lon/speed/accuracy shape.

→

awareness. Accepts or rejects each fix, accumulates distance and time, detects kilometre crossings.

→

the GPX writer and the screen-stays-on lock in parallel with the metrics engine.

→

the controller's state changes. No polling, no manual refresh calls.

## Why the layers are separated

### core/location

The *only* place that imports the GPS plugin. Swapping GPS providers, or feeding it recorded test routes, touches one file.

### domain/\*

Plain Dart — no Flutter, no UI import allowed. This is what makes the maths in §3 unit-testable in isolation, run in milliseconds, with no phone or simulator needed.

### features/live\_run

The screen and its controller. Reads from `domain`, never the reverse — the calculation layer has no idea a screen exists.

This isn't architectural tidiness for its own sake — it's what let the outlier-rejection logic in §3 be tested against thousands of synthetic GPS traces with zero live GPS involved, and what let this session catch and fix three separate bugs before they were ever tested against your own runs.

## 03 How each metric is actually calculated

The phone hands the app a raw stream of (latitude, longitude, timestamp, accuracy) readings. Everything you see on the tracking screen is derived from that stream — nothing is estimated from your step count or phone motion sensors.

## How often a fix arrives

| REQUESTED INTERVAL | REQUESTED PRECISION         | DISTANCE FILTER      | TYPICAL REAL INTERVAL     |
|--------------------|-----------------------------|----------------------|---------------------------|
| 1 s (Android)      | <b>best</b> -for-navigation | 0 m — no fix skipped | ~1–3 s, terrain-dependent |

One second is a *request*, not a guarantee — the OS and the GPS chipset decide the real cadence, and it varies with sky visibility. Open sky and a good lock: close to 1 fix/second. Under tree cover, between tall buildings, or on a cold start: gaps of several seconds are normal, and the app is built to tolerate them without the display freezing or jumping.

## The core geometry

Every distance calculation starts with the **haversine formula** — the standard way to find the straight-line distance between two latitude/longitude points on a sphere. Given any two consecutive fixes, it returns the distance between them in metres; dividing that by the time between their timestamps gives the implied speed for that segment.

### Current speed

km/h · min/km

```
distance(oldest, newest) ÷ time  
– over the last 3 seconds of fixes
```

A short rolling window, not an instant reading — smooths out single-fix jitter. If fixes arrive slower than every 3 s (weak signal), it falls back to just the last two points rather than freezing.

### Average speed

km/h · min/km

```
total distance ÷ total moving time  
– since Start, excluding pauses
```

Cumulative across the whole run. A discarded bad fix (see §4) contributes nothing here — the average only reflects distance the engine actually accepted.

### Distance

km, 2 d.p.

```
Σ haversine(fixn, fixn+1)  
– summed segment by segment
```

Built incrementally, one fix at a time — the app never needs to replay the whole run's history to update this number.

### Split pace

current km in progress

```
distance into this km ÷ time  
into this km, live
```

Resets each time you cross a whole kilometre. The crossing moment itself is *interpolated* —

see below — so it doesn't wait for the next GPS fix to land exactly on the line.

## Last split

m:ss for that km

time at which the previous  
km boundary was crossed

Because the crossing time is interpolated between two real fixes, this is accurate to a fraction of a second — not rounded to whichever fix happened to land nearest the line.

## Elapsed time

h:mm:ss

$\Sigma$  time between accepted  
consecutive fixes

**Not** a wall clock. Pausing genuinely stops this counter — the gap while paused is never added, however long you stand still.

**On that interpolation:** say you're at 970 m at one fix and 1,050 m at the next, one second later. The 1 km mark falls 75% of the way through that segment, so the app estimates you crossed it 750 ms into that one-second gap — rather than crediting the whole split to whichever fix happened to land on either side of the line. This is also why a split time can differ slightly from what a simpler tracker would show for the identical GPS data.



## 04 Precautions against bad GPS data

GPS receivers occasionally hand back a wrong answer — most often right at the start of a run, before the chipset has a confident lock, or indoors, where reflected signal ("multipath") can make a stationary phone look like it's slowly moving. Every fix passes through several checks before it's allowed to count, and the checks differ slightly for running versus cycling.

### GUARD

### WHAT IT CATCHES

### THRESHOLD

#### Measured-accuracy filter

Some fixes carry an accuracy figure the OS never actually measured — just a placeholder. Trusting that value as if it were a confident

reject if **accuracy unmeasured**

| GUARD                  | WHAT IT CATCHES                                                                                                                                                                                                                                                                                                                                                                           | THRESHOLD                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
|                        | reading would let an arbitrarily bad fix straight through the next filter, so it's rejected outright, regardless of what the number says.                                                                                                                                                                                                                                                 |                                                                                                          |
| Accuracy filter        | The OS reports a confidence radius with every fix. A fix the phone itself isn't confident about is dropped before it reaches any calculation.                                                                                                                                                                                                                                             | reject if <code>accuracy &gt; 25 m</code>                                                                |
| Plausible-speed filter | No one sustains this pace for the chosen activity. A fix implying it is treated as a glitch — a stale or default location before the GPS gets a real lock — not real motion. Cycling's cap is higher, since a fast descent is routine on a bike but not on foot.                                                                                                                          | reject if implied speed > <code>12 m/s</code> (43 km/h) running<br><code>25 m/s</code> (90 km/h) cycling |
| Absolute-jump cap      | Closes a loophole in the speed filter alone: given a long enough gap since the last fix, <i>any</i> distance starts to look like a slow, plausible pace. This catches a large teleport regardless of how much time has passed. Cycling's cap is larger, since a bike can cover far more ground in a genuinely sparse-fix gap (a tunnel, a backgrounded phone) without it being a bad fix. | reject if jump > <code>20 km</code> running · <code>40 km</code> cycling                                 |
| Noise floor            | Indoors, reflected/multipath fixes routinely wander a few metres between updates while the phone doesn't move at all — well within even the plausible-speed filter's limits, so that check alone never catches it. A segment this small is treated as noise: the app keeps tracking your position but credits no distance or time for it, the same as while paused.                       | ignore if <code>segment &lt; 3 m</code>                                                                  |

**Straightness filter**

The noise floor alone isn't enough for every kind of indoor drift: multipath can also creep steadily in one direction for several seconds — each individual step clearing the noise floor, even looking like a plausible running speed — before snapping back close to where it started. Real motion's net displacement over a few seconds stays close to the total ground it covered; a drift-then-snap-back's doesn't, because the eventual reversal cancels much of it out. A segment only counts once the recent path is straight enough to trust.

ignore if  $\text{net} \div \text{path} < 0.7$   
over last 25 s

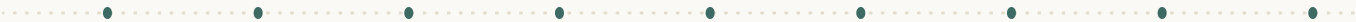
**What happens to a rejected fix.** It's discarded from the live metrics — distance and average speed act as though it never arrived. But an implausible-speed or absolute-jump rejection isn't simply forgotten: if the *next* fix agrees with the rejected one (i.e. two consecutive readings that are mutually consistent, showing at least a couple of metres of real movement between them), the app concludes the earlier "good" fix was actually the mistake, and quietly re-anchors its tracking onto the new, more plausible position — crediting no extra distance or time for the jump itself, the same way a pause and resume works. That candidate has to be confirmed within 30 seconds, or it's treated as a genuine gap rather than a recoverable glitch.

That recovery step exists because the obvious alternative — always trusting the last accepted fix — has a sharp failure mode: if that fix turns out to have been the bad one, *every subsequent real fix* near your actual position also looks like an impossible jump and gets rejected too, until enough time passes for the maths to catch up. An early version of this logic quarantined 77 seconds of perfectly good running data this way before the fix below it.

**On the straightness filter specifically:** no check that only looks at position can spot a drift-then-snap-back *before* it snaps back — up to that point, a steady drift in one direction is genuinely indistinguishable from someone walking that way. In practice this means the very first such episode after a stable period can credit a small amount of

phantom distance (typically under 15 m, from real indoor test captures), but the moment the drift reverses, that reversal itself is rejected and stays in view for the next 25 seconds — so a second drift-and-snap-back cycle right afterwards, of the kind that previously produced several seconds of a falsely elevated speed reading, is caught immediately and credits nothing further.

The GPX file written to disk, by contrast, keeps **every** fix, filtered or not — filtering is a display decision the app makes for you live; the underlying recording stays complete, in case you ever want to re-process it differently.



## 05 What to expect in the real world

GPS accuracy on a phone is a function of sky visibility, not the app. These are realistic bands for a modern smartphone, not lab conditions.

### OPEN SKY · CLEAR CONDITIONS

Accuracy typically  **$\pm 3\text{--}8$  m**. Speed and pace settle within a few seconds of starting and track smoothly. This is the everyday case for most road and park running.

### URBAN CANYON · TALL BUILDINGS

Signal reflects off buildings before reaching the phone ("multipath"), inflating accuracy to  **$\pm 15\text{--}30$  m**. Pace readings get noisier; more fixes sit near the 25 m accuracy cutoff and get dropped.

### TREE COVER · NARROW TRAILS

Fix interval stretches — expect gaps of **3–8 s** between updates rather than a steady 1 s. Distance still accumulates correctly; the current-speed reading updates less often.

### FIRST 10–20 SECONDS OF ANY RUN

The GPS chipset is still narrowing its lock. This is precisely the window the outlier filters in §4 exist for — an early bad fix here is the most common real-world trigger.

**A rule of thumb:** if a single split or momentary speed reading looks implausible, distance and average speed are almost always still correct — the filters described in §4 exist



specifically to protect those cumulative numbers, even when one input sample was briefly wrong. It's the instantaneous readings — current speed, in-progress split pace — that are most exposed to real-time GPS noise, by nature of updating every second rather than averaging over the whole run.